Go Runtime Primer

Go ランタイムの source を読むために必要な前提知識を、`Scheduler` と `Stack` を主線に整理した学習用ノートです。Go を日常的に書いているが `runtime/proc.go` や `runtime/stack.go` はまだ重く感じる、という段階から読み始められるように構成しています。

> **編集注記**
> この文書は Go 公式文書や `runtime` source の逐語訳ではありません。`go.dev` 上の一次資料をもとに、概念の接続が見えやすい順に再構成した版です。実装の全網羅ではなく、「どこから読めば runtime が怖くなくなるか」を主目的にしています。

目次

Contents

はじめに 2
- この教材の主題 3
- Go ランタイムを読むと何が嬉しいのか 3
- 前提と範囲 4
- 2026 年 3 月 10 日時点の注意 4
- 参考にする一次資料 4
- 各章の見取り図 5

Go ランタイムを読む前の前提整理 5
- `slice`、`string`、`interface` を実体として捉える 5
- `unsafe.Pointer` は何を破るのか 6
- escape analysis が runtime と繋がる理由 6
- `defer`、`panic`、`recover` 6
- Go のメモリモデル 6
- thread と stack の最小前提 6
- calling convention と Go assembler 7
- `go:systemstack` の意味 7
- `netpoll` と syscalls を理解する最低限 7
- ここで押さえるべきキーワード 7
- 次章への橋渡し 8

`Scheduler` と `Stack` 8
- `G/M/P` とは何か 8
- なぜ `P` が必要なのか 8
- run queue 8
- `schedule` と `findRunnable` 9
- work stealing 9
- syscall で止まると何が起きるか 9
- preemption 9
- goroutine stack は小さく始まる 9
- `morestack` と stackguard 10
- system stack 10

stack growth と pointer 10
runtime.HACKING.md の位置づけ 10
scheduler と stack を一緒に学ぶ理由 10
次章への橋渡し 11
観測しながら理解する 11
go tool trace 11
何を見るとよいか 11
GODEBUG=schedtrace 12
pprof 12
runtime/metrics 12
短い観測用プログラムを持つ 12
stack を観測するには 12
trace と source の接続 13
何を観測したら読み始めるか 13
次章への橋渡し 13
source reading guide 13
1. runtime/HACKING.md 13
2. runtime/proc.go 14
schedule をどう読むか 14
findRunnable をどう読むか 14
runtime/stack.go 14
morestack の見方 15
3. malloc.go 15
4. GC 周辺 15
実際の読書順 15
読み方のコツ 16
この段階で説明できるようになりたいこと 16
次章への橋渡し 16
付録と次の一歩 16
章ごとの復習ポイント 16
小さな観測課題 16
allocator と GC の最小見取り図 17
Go 1.26 との差分の見方 17
用語小事典 17
よくあるつまずき 17
P がなぜ必要か腹落ちしない 17
goroutine stack と system stack の違いが曖昧 18
trace を見ても何が重要か分からない 18
GC が難しすぎる 18
次に読むなら 18
参考資料 18
おわりに 18

はじめに

この文書は、Go ランタイムの source を読むための足場を作るノートです。目標は、`runtime/proc.go` や `runtime/stack.go` を最初から全部理解することではありません。どの概

念がどのファイルに対応していて、どこを読むと何が分かるのかを一本の線にすることです。

Go を普段使っていると、goroutine、channel、`go test`, `pprof`, `trace` のような道具は身近でも、その下で動いている scheduler や stack 管理は黒箱に見えがちです。しかしランタイムを読むと、黒箱の中身は「非常に大きな魔法」ではなく、いくつかの明確な責務の組み合わせだと分かります。どの goroutine をどの thread で走らせるか。stack をどう増やすか。GC と scheduler はどう協調するか。システムコールで止まった thread をどう扱うか。これらはすべて runtime の責務です。

この教材の主題

本書の主線は `Scheduler` と `Stack` です。理由は単純で、ここが見えると Go ランタイム全体の見取り図が急に読みやすくなるからです。GC や allocator ももちろん重要ですが、それらを深く読む前に、goroutine がどうスケジュールされ、どこで止まり、どの stack で動いているのかが分かっていたほうが全体像を掴みやすいです。

この本では、次の流れで進みます。

1. まず、Go ランタイムを読むために最低限必要な言語とメモリモデルの知識を整理する
2. 次に、OS thread、stack、ABI の最低限を押さえる
3. そのうえで、`G/M/P`、run queue、preemption、system stack をまとめて理解する
4. さらに、`trace`、`pprof`、`runtime/metrics`、`GODEBUG` でそれらを観測する
5. 最後に `runtime/HACKING.md`、`proc.go`、`stack.go` へ入る読書ガイドを示す

つまり「概念の説明」と「source の入口」と「実際の観測」を分断しません。読んだ概念が手元のプログラムでどう見えるかまで一度に繋ぎます。

この章の到達点

本書を読み終えるころには、少なくとも次を説明できる状態を目指します。

- goroutine が OS thread と 1:1 ではない理由
- `G/M/P` のそれぞれが何を表すか
- goroutine stack と system stack の違い
- `schedule` と `findRunnable` が何をしているか
- `go tool trace` や `GODEBUG=schedtrace` を見て scheduler の混雑を読む基本

Go ランタイムを読むと何が嬉しいのか

最初の利点は、パフォーマンス問題の切り分けが良くなることです。goroutine を大量に立てたとき、どこで詰まっているのか。システムコールが多いとき、scheduler はどう振る舞うのか。GC がどこで停止し、どの程度 background worker が動いているのか。こうしたことは、ランタイムの構造を少し知るだけで trace の見え方が大きく変わります。

次の利点は、`unsafe` や `cgo`、プロファイラ、トレース、低レイヤの同期原語の見え方が良くなることです。たとえば `go:systemstack` がなぜ存在するのか、write barrier がなぜ compiler と runtime の両方にまたがるのか、panic 中にどんな stack が使われるのか、といった疑問に筋の通った答えが持てるようになります。

最後に、言語処理系としての Go の一体感が見えてきます。Go は仕様、コンパイラ、ランタイム、標準ライブラリの境界が比較的よく揃っている言語です。runtime を読むと、

「goroutine は文法糖ではなく runtime の実体に支えられた概念だ」ということがよく分かります。

前提と範囲

読者には、Go の基本文法、goroutine、channel、`sync.Mutex`、`context`、`pprof` の存在くらいは知っていることを想定します。ただし `interface` の実体や `unsafe.Pointer`、escape analysis までは曖昧でも構いません。本書で runtime 読解に必要な範囲だけ整理します。

一方で、範囲は絞ります。次は本編の主題にはしません。

- GC 実装の細部の完全読解
- `compiler` と `ssa` パッケージの内部
- `cgo` の全体像
- signal 処理の全細部
- `Green Tea` GC の設計を source ベースで追い切ること

GC と allocator は本編後半で概観しますが、それは `proc.go` と `stack.go` を読むための補助として扱います。本書のゴールは runtime の全網羅ではなく、「まず何が重要か」を固めることです。

2026 年 3 月 10 日時点の注意

Go の runtime は継続的に変わります。特に GC まわりは版ごとの差分が大きく、2026 年 3 月 10 日時点では Go 1.26 の `Green Tea` GC が既定になっています。一方、`GC Guide` は本文中で Go 1.19 時点の GC を説明していると明記されています。したがって GC を理解するときは、「guide で原理を掴む」「release notes で最新差分を補う」を分ける必要があります。

この本では、その違いを曖昧に混ぜません。scheduler と stack は現行 source の読み方を中心に、GC は「今読むために必要な足場」と「差分注記」を分けて整理します。

> **注意**
> runtime source は版ごとに細部が変わります。本書の狙いは行番号を暗記することではなく、`proc.go` や `stack.go` を読んだときに「今はこの責務の話をしている」と分かることです。関数名や構造は追えても、版差分にはある程度揺れがある前提で読むのがよいです。

参考にする一次資料

この文書は `go.dev` 上の公式資料を主参照にします。具体的には次です。

- `The Go Memory Model`
- `A Quick Guide to Go's Assembler`
- `Diagnostics`
- `runtime/HACKING.md`
- `runtime/proc.go`
- `runtime/stack.go`
- `GC Guide`
- `Go 1.26 Release Notes`

これらをそのまま並べると少し散らばって見えますが、実際には互いに強く接続しています。本書ではその接続順を先に示します。

編集注記
ランタイム読解の最初の失敗は、「とりあえず `proc.go` を開く」ことです。もちろん最終的にはそうしますが、前提のない状態で読むと `G/M/P`、preemption、system stack、sysmon、netpoll が一度に出てきて苦しくなります。本書は、その苦しさを減らすための導入です。

各章の見取り図

以下の順で進みます。

1. Go の値表現、メモリモデル、OS/ABI の最低限を整理する
2. `G/M/P`、run queue、preemption、stack growth をまとめて理解する
3. `trace`、`schedtrace`、`runtime/metrics` で手元のプログラムから観測する
4. `runtime/HACKING.md` から `proc.go`、`stack.go` へ入る順序を示す
5. 補助として allocator と GC の最小見取り図を載せる

この順序には理由があります。runtime の source は、概念だけ見ても実感が湧きにくく、逆に source だけ見ても文脈が足りません。観測を中間に挟むことで、概念と source の橋を架けます。次章ではそのための前提として、Go の値表現、メモリモデル、OS/ABI の最低限から入ります。

Go ランタイムを読む前の前提整理

runtime source を読むときに最初につまずくのは、scheduler そのものより前提知識のズレです。goroutine は知っていても、`interface` の実体、escape analysis、memory model、system stack の意味が曖昧だと、個々のコードは追えても全体像が繋がりません。この章では、runtime 読解に必要な前提だけを絞って整理します。

この章の到達点
この章では次を押さえます。

- Go の値が「見た目通りの箱」ではなく、runtime のデータ構造に支えられていること
- `happens-before` と同期原語の意味を、runtime の視点で捉え直せること
- thread、stack、calling convention の最低限が `proc.go` と `stack.go` 読解の前提になること

`slice`、`string`、`interface` を実体として捉える

普段 Go を書くとき、`slice` や `string` や `interface` は高水準な値として扱います。しかし runtime を読むには、それらがどういう形の値として存在しているかを意識したほうがよいです。

- `slice` ポインタ、長さ、容量を持つ
- `string` ポインタと長さを持つ
- `interface` 型情報とデータへの参照を持つ

これらを理解しておくと、stack growth や write barrier や escape analysis がなぜ必要かが見えやすくなります。値の見た目が高水準でも、runtime は最終的にポインタとメタデータの整合性を管理しているからです。

`unsafe.Pointer` は何を破るのか

runtime source では `unsafe` が頻繁に出てきます。これは単なる低レイヤ好みではなく、通常の型システムだけでは表しにくい内部データ構造や ABI 境界を扱う必要があるからです。`unsafe.Pointer` を理解するうえで重要なのは、「何でも自由にしてよい」ではなく、「GC と型追跡の前提を自分で保証する」ことです。

ランタイム内部では、ポインタの生存、stack 移動時の更新、write barrier の適用など、GC と深く結びついた事情があります。したがって `unsafe` の意味は、C 風の生ポインタ操作というより、**GC と協調するための危険な抜け道** と捉えるほうが実情に近いです。

escape analysis が runtime と繋がる理由

escape analysis はコンパイラの話に見えますが、runtime 読解にも重要です。変数が stack に置かれるのか heap に逃がされるのかで、GC の対象になるか、pointer map の扱いがどう変わるか、stack growth 時にどう更新されるかが変わるからです。

この点を押さえると、「なぜこの値は heap にあるのか」「なぜ stack 上の一時オブジェクトが GC と関係するのか」が見えやすくなります。コンパイラの判断が runtime の責務に直結している、という接点です。

`defer`、`panic`、`recover`

`defer`、`panic`、`recover` は表面上は言語機能ですが、runtime の支えなしには成立しません。特に stack unwinding、panic chain、system stack への切り替え、reporting などは runtime の仕事です。これらの細部を今すぐ追う必要はありませんが、「制御フローが通常の return だけで進むわけではない」という前提は持っておく必要があります。

これは stack の章で効きます。stack は単なる配列ではなく、panic、preemption、GC scan など、多くの機構の舞台だからです。

Go のメモリモデル

`The Go Memory Model` を読む理由は、アプリケーションの正しさだけではありません。runtime そのものが、どの同期で何を保証しているかを読むためです。`happens-before` を理解していないと、run queue への push/pop、状態遷移、atomic field の意味が見えにくくなります。

特に意識したいのは次です。

- goroutine 間の順序は自動では保証されない
- channel、mutex、atomic は順序保証の道具である
- scheduler が切り替えることと、メモリ順序が保証されることは別問題である

この 3 つが混ざると runtime のコードは読みづらくなります。スケジューリングと同期は別層であり、両方を区別して読む必要があります。

thread と stack の最小前提

Go runtime を読むには OS の thread モデルを最低限知っている必要があります。Go は goroutine を user-space scheduler で multiplex しますが、その下では依然として OS thread が実行主体です。さらに syscalls や cgo に入ると、thread のブロッキングや pinning が scheduler に影響します。

stack についても、ただ「関数呼び出しで積まれる領域」というだけでは足りません。特に意識したいのは次です。

- 各 thread は OS 由来の stack を持つ
- 各 goroutine は小さく始まる可動 stack を持つ
- runtime の一部処理は goroutine stack ではなく system stack で動く

この区別が見えるだけで、`stack.go` の見え方はかなり変わります。

calling convention と Go assembler

`A Quick Guide to Go's Assembler` を読む価値は、アセンブリを書くためだけではありません。prologue/epilogue、レジスタ、stack frame、ABI、`TEXT` 宣言、引数と戻り値の置き方が分かると、runtime source に出てくる低レベル部分が急に怖くなくなります。

特に stack growth や preemption では、関数の入口で stackguard を見る、必要なら `morestack` へ飛ぶ、といった処理が重要です。これは言語仕様だけ読んでいても見えません。最低限の ABI 感覚があると、なぜそのコードが必要かが理解しやすくなります。

`go:systemstack` の意味

runtime 内部では、goroutine stack で動くと危険な処理があります。たとえば stack growth の最中にさらに stack を必要とする処理、scheduler の核心、GC や signal に近い処理です。そうした場所では system stack を使う必要があります。

ここで system stack を「特別な別世界」と思わないことが大切です。要するに、「今の goroutine の可動 stack に依存すると危ないから、固定的で runtime 管理しやすい stack で動く」というだけです。この感覚を持つと `go:systemstack` が自然に見えます。

`netpoll` と syscalls を理解する最低限

Go の scheduler を理解するとき、syscall と I/O 待ちは避けて通れません。goroutine が syscall で止まると、その goroutine を動かしていた thread も詰まることがあります。そのとき scheduler は別 thread と `P` を使って他の仕事を進める必要があります。さらに network I/O は `netpoll` を通じて待ち合わせと再開が行われます。

本書では `netpoll` の source を深追いしませんが、「I/O 待ちは scheduler 設計に直結している」という認識は前提として持ちます。

ここで押さえるべきキーワード

この章を終える時点で、最低限次の言葉が曖昧でなければ十分です。

- `happens-before`
- `atomic`
- goroutine stack
- system stack
- escape analysis
- `unsafe.Pointer`
- ABI
- syscall block

これだけでも、次章の `G/M/P` や stack growth の話にかなり入りやすくなります。

> **注意**
> ここで挙げた前提を完璧にする必要はありません。重要なのは、runtime source を読んで

いて「あ、これはメモリモデルの話だ」「ここは system stack の話だ」とラベル付けできることです。完全な予習より、概念の見出しを持っておくほうが効きます。

次章への橋渡し

ここまでで、Go の値表現、メモリモデル、OS/ABI の最小前提は揃いました。次章ではいよいよ `G/M/P`、run queue、preemption、goroutine stack、system stack をまとめて見て、`proc.go` と `stack.go` の見取り図を作ります。

`Scheduler` と `Stack`

Go ランタイムを読むうえで最初の核は `Scheduler` と `Stack` です。goroutine は OS thread と 1:1 ではありませんし、stack も固定長ではありません。この二つが分かると、`proc.go` と `stack.go` の大半は「巨大な謎」ではなく、「複数の制約を解くための設計」に見えてきます。

この章の到達点

この章では次を理解することを目標にします。

- `G/M/P` の役割分担
- run queue と work stealing の意味
- preemption と syscall block が scheduler にどう効くか
- goroutine stack と system stack の違い
- stack growth と `morestack` の目的

`G/M/P` とは何か

Go runtime を初めて読むと、`G`, `M`, `P` という一文字名に戸惑います。意味は次の通りです。

- `G` 実行対象そのもの。goroutine の状態、stack、実行位置などを持つ
- `M` machine の略で、OS thread に対応する実行主体
- `P` processor の略で、実行に必要なローカル資源の束

重要なのは `P` です。多くの初学者は `G` と `M` だけでよさそうに見えますが、Go runtime は `P` を介することで、ローカル run queue、allocator cache、timer などの資源を thread から少し切り離しています。これにより work stealing や syscall 中の資源の受け渡しが整理しやすくなります。

なぜ `P` が必要なのか

もし goroutine を thread へ直接束ねるだけなら、local queue や cache も thread ごとに持てそうです。しかし thread は syscall や cgo で簡単にブロックします。そのたびに実行資源まで一緒に止まると、他の goroutine を動かしづらくなります。

そこで `P` を独立資源として持ち、thread が止まったら `P` を別の `M` へ渡せるようにします。これが Go runtime の非常に重要な発想です。OS thread を実行器、`P` を実行許可とローカル資源の単位として分けることで、goroutine scheduler を柔軟にしています。

run queue

各 `P` はローカル run queue を持ちます。新しく runnable になった goroutine は、まずどこかの `P` の local queue へ入ります。これにより lock 競合を減らし、近くの仕事を近くで回しやすくします。全体共有 queue もありますが、常にそこを主に使うわけではありません。

この構造は、CPU scheduler や work stealing runtime でよく見る設計と似ています。共有 queue を 1 個だけにすると競合が増え、ローカルだけだと負荷が偏ります。そこで local queue を基本にしつつ、足りないときは stealing で均します。

`schedule` と `findRunnable`

`proc.go` を読むときの入口は `schedule` と `findRunnable` です。細部は多いですが、役割を荒く見ると次のようになります。

- `schedule` 今この `M` が次にどの `G` を走らせるか決める中心ループ
- `findRunnable` local/global queue、netpoll、timer、steal などを見て runnable な仕事を探す

この二つを先に押さえると、周辺の関数が「どこで新しい `G` を enqueue し、どこでそれを拾うか」という関係で見えてきます。逆にここを押さえないと、個々の helper 関数が何のためにあるのか見えにくくなります。

work stealing

ある `P` が暇で、別の `P` が多くの runnable goroutine を抱えているなら、仕事を盗んだほうが全体が進みます。これが work stealing です。Go runtime は local queue を基本にしつつ、必要に応じて他の `P` から仕事を分けてもらいます。

この仕組みがあるおかげで、goroutine 生成が偏っても scheduler 全体が極端に崩れにくくなります。もちろん万能ではありませんが、thread ごとの固定割り当てよりずっと柔軟です。

syscall で止まると何が起きるか

goroutine が syscall に入ると、その goroutine を実行していた `M` も OS 側で止まり得ます。しかし runtime は全体を止めたくありません。そこで、その `M` が持っていた `P` を切り離して別の `M` へ回し、他の runnable goroutine を進められるようにします。

ここで `P` の分離が効きます。もし thread と実行資源が一体なら、syscall で thread が止まるたびにスケジューリング資源も失われます。`P` を介しているからこそ、Go は blocking syscall と user-space scheduling を両立しやすくなっています。

preemption

goroutine を cooperative に走らせるだけだと、長く CPU を握り続ける goroutine が他の goroutine を飢えさせる危険があります。そこで runtime は preemption を入れ、必要なら途中で実行権を返してもらいます。

Go の preemption は OS の割り込みと完全に同じではありません。safe point や stack check と協調しながら、runtime が goroutine を止めやすい場所で止めます。この「どこでも即停止できるわけではない」という感覚は重要です。GC の停止や stack scan も safe point と深く関係するからです。

goroutine stack は小さく始まる

各 goroutine に最初から巨大 stack を割り当てると、軽量 thread としての利点が薄れます。そこで Go は小さな stack から始め、必要になったら成長させます。これが goroutine stack の設計です。

固定 stack ではなく可動 stack を持つということは、runtime が stack growth、stack copy、pointer 更新、GC scan を扱う必要があることを意味します。つまり stack は単なるメモリ領域ではなく、runtime の主要管理対象です。

`morestack` と stackguard

関数呼び出しの入口では、今の stack に十分な空きがあるかを確認する必要があります。足りなければ `morestack` のような経路へ入り、stack を増やしてから再実行します。これが `stackguard` チェックの基本的な意味です。

ここで面白いのは、stack check が「関数 prologue の一部」として言語実装に組み込まれていることです。つまり compiler と runtime が協調しないと成立しません。runtime だけ見ても、なぜ `morestack` が必要かは半分しか分かりません。

system stack

goroutine stack は grow し得る可動 stack です。一方で runtime の一部処理は、その上で動くと危険です。たとえば stack growth の途中でさらに stack growth が必要になるような処理、scheduler の重要部分、signal や GC の一部などです。そのため runtime は system stack を使います。

system stack を理解するときは、「goroutine stack より偉い stack」ではなく、「runtime が自分自身の足場を確保するための固定 stack」と捉えるのがよいです。`go:systemstack` は、その足場へ明示的に切り替える印です。

stack growth と pointer

stack が成長するとき、ただ新しい大きい領域を確保してバイト列をコピーすれば終わりではありません。stack 上のポインタやフレーム境界、GC が見る pointer map などと整合が取れていなければいけません。ここで runtime と compiler の協調が再び重要になります。

この点が見えると、stack は「各 goroutine にくっついた単純な配列」ではなく、「GC と ABI と scheduler の接点」として見えてきます。

`runtime.HACKING.md` の位置づけ

`runtime/HACKING.md` は source を読む前の見取り図として非常に有用です。`G/M/P`、goroutine stack、system stack、malloc ルール、write barrier 禁止領域など、runtime 独特の常識が短くまとまっています。本書で先にこれらを文章で整理しているのは、まさにこのファイルを読みやすくするためです。

> **編集注記**
> `proc.go` の中身を先に全部追いかけるより、「`P` はローカル資源の束」「syscall 中は `P` を他へ渡せる」「goroutine stack は grow する」「危ない処理は system stack でやる」という 4 点を固めたほうが、結果的に読む速度が上がります。

scheduler と stack を一緒に学ぶ理由

この二つは別の話に見えて、実際には強く結びついています。preemption は stack check と関係し、GC scan は stack map と関係し、system stack への切り替えは scheduler と深く関係します。したがって「まず scheduler だけ」「あとで stack」はあまり効率がよくありません。本書で同じ章に置くのはそのためです。

> **注意**
> runtime source の変数名や helper 関数は版ごとに変わることがあります。しかし `G/M/P` の

責務分担、stack growth、system stack、syscall block といった設計の核は比較的安定しています。まずは核を掴むのが先です。

次章への橋渡し

ここまでで概念の見取り図はできました。次章では `trace`、`pprof`、`runtime/metrics`、`GODEBUG=schedtrace` などを使って、scheduler と stack の振る舞いを実際のプログラムから観測する方法を整理します。source を読む前に観測を挟むことで、概念に手触りを持たせます。

観測しながら理解する

runtime の概念は、文章だけで理解しようとすると少し乾きやすいです。scheduler も stack も、実際にプログラムを走らせたときにどう見えるかを一度観測すると理解が定着しやすくなります。この章では `go tool trace`、`pprof`、`runtime/metrics`、`GODEBUG` を使って、runtime の振る舞いを手元で観測するための最小導線をまとめます。

この章の到達点

この章では次を押さえます。

- `trace` は scheduler の時系列を見る道具だということ
- `pprof` は資源消費の偏りを見る道具だということ
- `runtime/metrics` は定常状態の指標を見る入口だということ
- `GODEBUG=schedtrace` は scheduler の荒い健康診断として便利だということ

`go tool trace`

`trace` は runtime の時系列イベントを追うための道具です。goroutine の生成、blocking、syscall、network wait、GC、STW などが時間軸で見えます。scheduler を理解したいときに最も強い入口で、`G/M/P` の動きや goroutine の詰まり方を視覚的に捉えやすいです。

実際には、短いプログラムへ `runtime/trace` を仕込み、出力を `go tool trace` で開きます。

```
f, _ := os.Create("trace.out")
trace.Start(f)
defer trace.Stop()
```

この一手間で、「goroutine を 1000 個立てたが本当に同時に進んでいるのか」「syscall 中に他の goroutine はどう動いたか」といった疑問が時間軸で見えるようになります。

何を見るとよいか

最初は細かいイベント名を全部追う必要はありません。まず見るべきは次です。

- goroutine が runnable から running へ移る流れ
- syscall へ入ったときの thread 側の変化
- GC 周期の始まりと終わり
- thread 数と runnable goroutine の関係

この観点だけでも、scheduler が単に round-robin しているのではなく、かなり多くの状態遷移を扱っていることが見えてきます。

`GODEBUG=schedtrace`

`schedtrace` は、一定周期で scheduler の概要を stderr へ吐く簡易観測です。trace ほど視覚的ではありませんが、セットアップが軽く、手元の小さな検証には非常に便利です。

```
GODEBUG=schedtrace=1000,scheddetail=1 go run main.go
```

この出力からは、`gomaxprocs`、idle `P`、run queue の長さ、thread 数などをざっくり見られます。まずは「暇な `P` があるのに runnable goroutine が多いのか」「thread が増えすぎていないか」といった荒い健康診断に使うとよいです。

`pprof`

`pprof` は CPU 時間やメモリ使用量の偏りを見る道具です。scheduler そのものの状態遷移を直接見るわけではありませんが、「runtime にどれくらい時間を使っているか」「GC や allocator にどの程度コストが乗っているか」を知るのに役立ちます。

特に次の観点が useful です。

- CPU profile で runtime 関数の比率を見る
- heap profile で allocation の偏りを見る
- goroutine profile で block している場所を見る

runtime source を読む前に `pprof` を一度触っておくと、「この関数は trace ではここに見え、profile ではこう見える」と多面的に理解しやすくなります。

`runtime/metrics`

`runtime/metrics` は、GC、scheduler、memory、assist などの定常指標を programmatic に取るための入口です。trace や profile が一回の観測の道具だとすれば、metrics は継続的な状態把握に向いています。

scheduler の観点では、「goroutine 数」「GC worker の挙動」「heap と stack の大きさ」といった数字を、過度に runtime source へ潜る前にざっくり把握できます。これにより、source 中の構造体や counters が「何のために存在するか」を実感しやすくなります。

短い観測用プログラムを持つ

runtime 読解を進めるなら、短い観測用プログラムをいくつか持っておくのが有効です。たとえば次のようなものです。

- goroutine を大量生成して channel 待ちさせる
- `time.Sleep` と busy loop を混ぜる
- network I/O を使って block/unblock を見る
- 深い再帰で stack growth を起こす

これらは本に大きな実装を載せる必要はありません。10〜20 行の小さな断片で十分です。重要なのは、「この現象を見たいからこのプログラムを走らせる」という対応があることです。

stack を観測するには

stack 自体は scheduler より視覚化しにくいですが、再帰や大きなフレームを持つ関数を使うと grow の気配が見えやすくなります。また panic 時の stack trace、goroutine dump、`pprof` の goroutine profile も役立ちます。

ここで意識したいのは、「stack の変化は常に目に見えるイベントとは限らない」ことです。scheduler のように timeline に出にくくても、panic trace、recursion、system stack への切り替えに注目すると、存在感が見えてきます。

`trace` と source の接続

観測の価値は、結果を source のどこへ戻せるかにあります。たとえば trace で runnable goroutine が長く溜まっているなら、`findRunnable` や run queue の取り回しを読みたくなります。深い再帰で stack に関連する挙動が気になるなら、`morestack` と `stack.go` を読みたくなります。

つまり観測は、「source を読む理由」を具体化するための道具です。本書で中盤に置いているのはそのためです。

何を観測したら読み始めるか

目安として、次のどれか一つでも観測できれば次章へ進めます。

- `trace` で goroutine の runnable/running/blocking を見た
- `schedtrace` の出力から `P` や run queue の存在を確認した
- 再帰や panic から stack の存在を意識できた
- `pprof` で runtime 側の関数が profile に現れるのを見た

全部を揃える必要はありません。大事なのは、`proc.go` や `stack.go` を「見たことのある現象の説明書」として読める状態にすることです。

> **注意**
> 観測ツールはどれもオーバーヘッドを持ちます。trace や profile の結果は、無計測時の実行と完全に同じではありません。ここでは絶対値より、状態遷移や偏りの傾向を見る道具だと理解しておくのが安全です。

次章への橋渡し

ここまでで、概念と観測の土台ができました。次章では、いよいよ `runtime/HACKING.md`、`proc.go`、`stack.go` をどの順で読むとよいか、関数単位の読書ガイドとしてまとめます。加えて `mallocgc` と GC まわりの最小入口も置きます。

source reading guide

この章では、Go ランタイム source を「どこから読むか」という順序を決めます。闇雲に `runtime` ディレクトリを開くと重すぎますが、入口を絞ればかなり読みやすくなります。本書では `runtime/HACKING.md` を最初の案内板にし、そこから `proc.go`、`stack.go`、最後に `malloc.go` と GC 周辺へ入る順序を取ります。

> **この章の到達点**
> この章では次を固めます。
>
> - 最初に読むべきファイルとその理由
> - `proc.go` のどの関数が scheduler の中心か
> - `stack.go` で何を追えばよいか
> - GC/allocator はどの粒度で触れれば十分か

1. `runtime/HACKING.md`

最初に読むべきなのは `runtime/HACKING.md` です。このファイルは runtime の内部常識を短くまとめたもので、`G/M/P`、goroutine stack、system stack、malloc まわりの作法、write barrier に触れてはいけない場面など、source を読む前に必要な前提が載っています。

ここでの読み方は「全部覚える」ではありません。次の見出しだけ先に押さえると十分です。

- `G`, `M`, `P`
- `stacks`
- scheduler に関する説明
- runtime code で特に注意するルール

このファイルを読むだけで、`proc.go` の一文字名が少し人間的になります。

2. `runtime/proc.go`

`proc.go` は scheduler の中心です。ただし最初から全部追う必要はありません。入口は次の関数で固定するとよいです。

- `schedule`
- `findRunnable`
- `execute`
- `newproc` 周辺
- syscall での出入りに関わる箇所

まずは「goroutine を作る」「runnable を探す」「実行する」「syscall で一時的に止まる」の流れだけを読むのがよいです。細かな timer や `netpoll` の枝にすぐ潜らず、中心線を先に作ります。

`schedule` をどう読むか

`schedule` は次の runnable goroutine を決めるループだと捉えます。重要なのは、そこに全ての policy が直接書いてあるわけではなく、実際の探索は `findRunnable` や run queue helper、steal、poll へ分散していることです。したがって `schedule` は「司令塔」、`findRunnable` は「仕事探し」と分けて読むのが整理しやすいです。

また、`schedule` は thread、`P`、goroutine の関係が最も露骨に見える場所でもあります。「なぜ `P` が必要か」は、この関数の前後を見るとかなり腑に落ちます。

`findRunnable` をどう読むか

`findRunnable` では、local queue、global queue、steal、timer、netpoll といった複数の候補から runnable な仕事を探します。ここは枝が多いので、最初は「仕事の供給源一覧」として読むのがよいです。つまり、

- 手元の `P` に仕事はないか
- 全体共有 queue にないか
- 他の `P` から steal できないか
- I/O 完了待ちから戻った仕事はないか

という順で読むと整理しやすいです。個々の分岐を暗記する必要はありません。

`runtime/stack.go`

`stack.go` の入口は、まず goroutine stack が grow し得ることを念頭に置くことです。そのうえで、次のテーマを追います。

- stackguard チェック
- `morestack`
- stack copy / grow

- system stack を使う場面

stack の章で重要なのは、単にコピー手順を追うことではありません。なぜ runtime が stack を自分で管理しなければならないか、なぜ system stack が必要かを source の中で確認することです。

`morestack` の見方

`morestack` は、stack が足りないときの逃げ道です。ここを読むと、compiler-generated prologue と runtime の接続がよく見えます。つまり stack growth は runtime 単体の話ではなく、関数呼び出し規約と結びついた設計だと分かります。

この視点は、ランタイム全体の読み方にも効きます。Go の機能は spec、compiler、runtime の境界で成立しているものが多く、stack はその典型です。

3. `malloc.go`

本書の主線は scheduler/stack ですが、runtime を読む以上 allocator の入口は触れておいたほうがよいです。`malloc.go` では `mallocgc` を中心に、size class、tiny allocator、`mcache` / `mcentral` / `mheap` の関係を「名前が怖くない程度」に整理します。

ここで目指すのは、malloc subsystem を完全に理解することではありません。`runtime` で allocation が発生したとき、どの層のキャッシュや共有構造が関わるのかを荒く掴むことです。

4. GC 周辺

GC は本書では補助章です。入口としては次を押さえれば十分です。

- `GC Guide` で mark/sweep と pacer の原理を掴む
- `mgc.go`、`mgcmark.go`、`mgcsweep.go` の責務分担を見る
- Go 1.26 の release notes で `Green Tea` GC の差分を確認する

重要なのは、`GC Guide` の説明対象が Go 1.19 時点であることを忘れないことです。guide は原理の足場として使い、最新版との差分は release notes と source で補います。

> **編集注記**
> GC をいきなり source から読むのはやや重いです。scheduler/stack よりも変数名や phase 分岐が多く、歴史的差分もあります。本書では「runtime の全体像を壊さない程度に読む」位置付けに留めます。

実際の読書順

本書のおすすめ順は次で固定します。

1. `runtime/HACKING.md`
2. `runtime/proc.go` の `schedule`
3. `runtime/proc.go` の `findRunnable`
4. `runtime/stack.go` の stack growth 関連
5. `runtime/malloc.go` の `mallocgc`
6. `GC Guide`
7. `runtime/mgc.go` 周辺
8. `Go 1.26 Release Notes`

この順は、概念が先に積み上がり、観測と source の対応が取りやすい順です。

読み方のコツ

runtime source は、最初から top-to-bottom に読む必要はありません。むしろ、次の読み方が有効です。

- 役割単位で読む
- 1 回で全部理解しようとしない
- trace や `schedtrace` と対応付ける
- 「これは scheduler の話」「これは stack の話」とラベルを持つ

この読み方をすると、ファイルの長さに圧倒されにくくなります。

この段階で説明できるようになりたいこと

この章まで来たら、少なくとも次が言えれば十分です。

- `proc.go` は scheduler の中心で、`schedule` と `findRunnable` が入口
- `stack.go` は goroutine stack と system stack の関係を見る場所
- `malloc.go` と GC 周辺は runtime 全体の補助線
- official docs と source は役割を分けて読むと進みやすい

> **注意**
> runtime source は内部 API なので、版が変わると関数の細部や helper の構成が変わります。したがって、「この関数のこの行が重要」と固定しすぎないほうがよいです。関数の責務を掴む読み方のほうが長持ちします。

次章への橋渡し

最後の章では、ここまでの復習、観測用の短い題材、GC/allocator の最小見取り図、用語集、次に読むべき source file をまとめます。

付録と次の一歩

本編では、Go ランタイムの中でも `Scheduler` と `Stack` を主線に、必要な前提知識と official source の入口を整理しました。この章では、復習、観測用の小課題、GC/allocator の最小見取り図、用語整理、次に読む source file をまとめます。

章ごとの復習ポイント

ここまでの内容を短く戻すと、各章の核は次のように整理できます。

- 導入: Go runtime は巨大な黒箱ではなく、scheduler、stack、GC、allocator、syscall/I/O 管理の責務の集合である
- 前提整理: `slice`、`interface`、memory model、ABI、system stack の意味を押さえると runtime source が読みやすくなる
- scheduler/stack: `G/M/P`、run queue、syscall block、preemption、stack growth が主線
- 観測: `trace`、`schedtrace`、`pprof`、`runtime/metrics` で runtime の挙動に手触りを持つ
- source guide: `runtime/HACKING.md` → `proc.go` → `stack.go` → `malloc.go`/GC の順で読む

この順序を持っているだけで、Go runtime の学習はかなり迷いにくくなります。

小さな観測課題

本編の理解を固めるなら、次の順で短い観測課題を試すとよいです。

1. goroutine を大量生成し、`GODEBUG=schedtrace=1000,scheddetail=1` で `P` と run queue を見る

2. `runtime/trace` を仕込み、sleep と busy loop を混ぜた program の時系列を見る
3. 再帰関数で stack growth を起こし、panic trace や profile を観察する
4. `runtime/metrics` で goroutine 数や heap/stack 系の指標を眺める
5. `pprof` で runtime 関数が CPU/heap profile に現れるのを確認する

これらは大きなサンプルである必要はありません。10〜20 行でも十分です。重要なのは、観測した現象を source のどこへ結びつけるかです。

allocator と GC の最小見取り図

本書では allocator と GC を主役にはしませんでしたが、runtime 全体像として最低限の見取り図は持っておくとよいです。

- allocator: `mcache` が各 `P` に近い小さなキャッシュとして働き、足りなくなると `mcentral` や `mheap` から補充される
- GC: mark/sweep と pacer によって、application goroutine と background worker が協調して回収を進める
- write barrier: pointer 更新が GC の正しさと結びつくため、compiler と runtime が協調する

この 3 点だけでも、`mallocgc` や `mgc*` 系ファイルを開いたときの心理的負荷がかなり下がります。

Go 1.26 との差分の見方

GC については `GC Guide` が Go 1.19 前提である一方、Go 1.26 では `Green Tea` GC が既定です。したがって読む順番としては、

1. `GC Guide` で mark/sweep、pacer、assist の原理を掴む
2. `Go 1.26 Release Notes` で差分を確認する
3. 必要に応じて `mgc.go` 周辺を見る

という順が安全です。guide を最新版の完全説明と見なさないことが重要です。

用語小事典

- `G` goroutine の実体
- `M` OS thread に対応する実行主体
- `P` 実行に必要なローカル資源の束
- run queue runnable goroutine の待ち行列
- work stealing 暇な `P` が他の `P` から仕事を盗む仕組み
- preemption 実行中 goroutine に途中で制御を返してもらう仕組み
- goroutine stack 小さく始まり grow する stack
- system stack runtime が危険な処理を行うための固定 stack
- `morestack` stack が足りないときの成長経路
- `netpoll` I/O 完了待ちと scheduler をつなぐ仕組み
- `write barrier` GC の正しさを保つための pointer 書き込み補助

よくあるつまずき

`P` がなぜ必要か腹落ちしない

thread と資源を分離し、syscall block 中でも実行資源を他へ回せるようにするためです。`P` を「CPU コアの抽象」ではなく「runtime ローカル資源の単位」と見ると理解しやすいです。

goroutine stack と system stack の違いが曖昧

goroutine stack は各 goroutine の実行用で grow し得ます。system stack は runtime が危険な内部処理を行うための固定的な足場です。

`trace` を見ても何が重要か分からない

最初は全部追わず、runnable/running/blocking、syscall、GC の 3 種類だけを見るとよいです。source へ戻すための観測なので、重要なラベルだけ拾えば十分です。

GC が難しすぎる

正常です。最初から GC source を全部追う必要はありません。scheduler と stack を先に固め、その後 `GC Guide` と release notes で足場を作ってから入るほうがよいです。

次に読むなら

本書の次に進む方向は大きく 3 つあります。

- scheduler を深める: `proc.go` の syscall, netpoll, timer の細部へ進む
- stack を深める: `stack.go`, asm, panic/unwind, signal handling を追う
- GC/allocator を深める: `mallocgc`, `mcache/mcentral/mheap`, `mgcmark`, `mgcsweep`, pacer を追う

Go runtime を「理解する」には、最初の一冊で全部やり切る必要はありません。主線を 1 本持っておくと、以後の枝がかなり読みやすくなります。

参考資料

- The Go Memory Model `https://go.dev/ref/mem`
- A Quick Guide to Go's Assembler `https://go.dev/doc/asm`
- Diagnostics `https://go.dev/doc/diagnostics`
- `runtime/HACKING.md` `https://go.dev/src/runtime/HACKING.md`
- `runtime/proc.go` `https://go.dev/src/runtime/proc.go`
- `runtime/stack.go` `https://go.dev/src/runtime/stack.go`
- GC Guide `https://go.dev/doc/gc-guide`
- Go 1.26 Release Notes `https://go.dev/doc/go1.26`

編集注記

Go runtime は source tree の規模こそ大きいですが、入口を間違えなければ十分に読める対象です。特に `Scheduler` と `Stack` を先に押さえる戦略は、GC や allocator を後から読むときにも効きます。

おわりに

Go runtime を読むために必要なのは、巨大な予備知識ではありません。何が scheduler の話で、何が stack の話で、何が GC や allocator の話かを見分けるラベルです。本書はそのラベルを揃えることを目指しました。

一度この地図を持つと、`proc.go` や `stack.go` はただ長いだけのファイルではなく、「goroutine がどう走り、どう止まり、どこで足場を切り替えるか」を説明する文書に見えてきます。そこまで来れば、あとは必要に応じて枝を伸ばしていけます。